

Chapter 7 메인 메모리

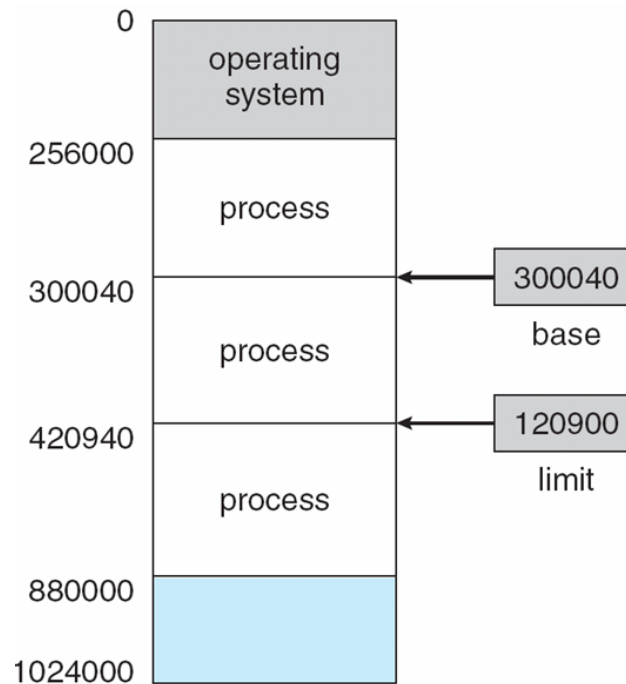
1. 배경
2. 스와핑 (Swapping)
3. 연속 메모리 할당
4. 세그먼테이션 (Segmentation)
5. 페이징 (Paging)
6. 페이지 테이블의 구조

7.1 배경

- ❑ 프로그램이 실행되기 위해서는 메모리로 로드되어(디스크로부터) 프로세스로 배치되어야 한다
- ❑ 메인 메모리와 레지스터
 - ➡ CPU가 직접 접근할 수 있는 유일한 저장장치이다
- ❑ 메모리 하드웨어
 - ➡ (주소, 읽기 요청) 또는 (주소, 쓰기 요청)만 수행
- ❑ 속도
 - ➡ 레지스터 접근에는 **1 CPU clock**이 (혹은 더 적은) 필요하다
 - ➡ 메인 메모리는 많은 사이클을 필요로 하고, CPU가 연산 수행을 하지 못하는 멈춤(**stall**) 현상이 발생
- ❑ 캐시(Cache)
 - ➡ stall 현상 해결
 - ➡ 메인 메모리와 CPU 레지스터 사이에 존재한다

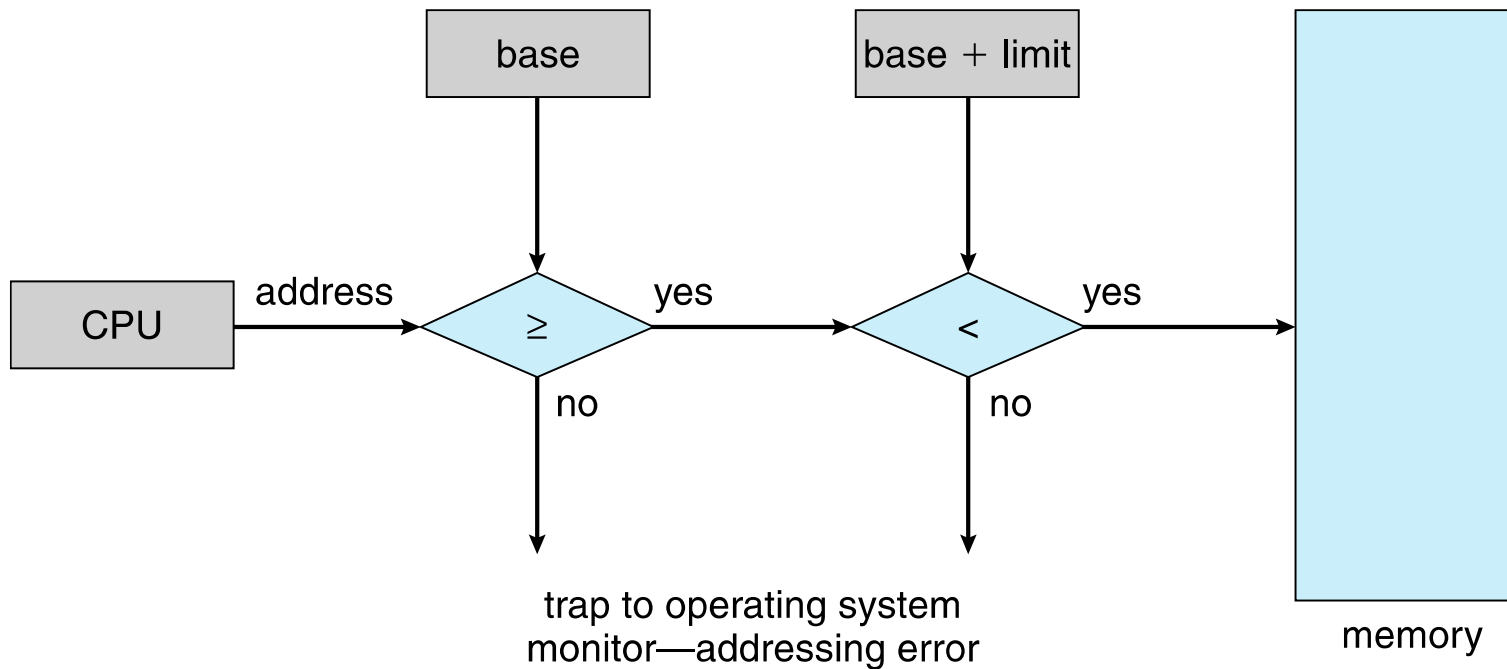
메모리 보호

- 사용자 프로세스로부터 커널 영역 또는 사용자 영역끼리의 보호가 필요
- 기준(**base**) 및 상한(**limit**) 레지스터 쌍이 논리 주소 공간을 정의
- CPU 하드웨어는 사용자 모드에서의 생성되는 모든 메모리 주소가 그 사용자의 기준과 상한 범위에 속하는 지 검사를 수행

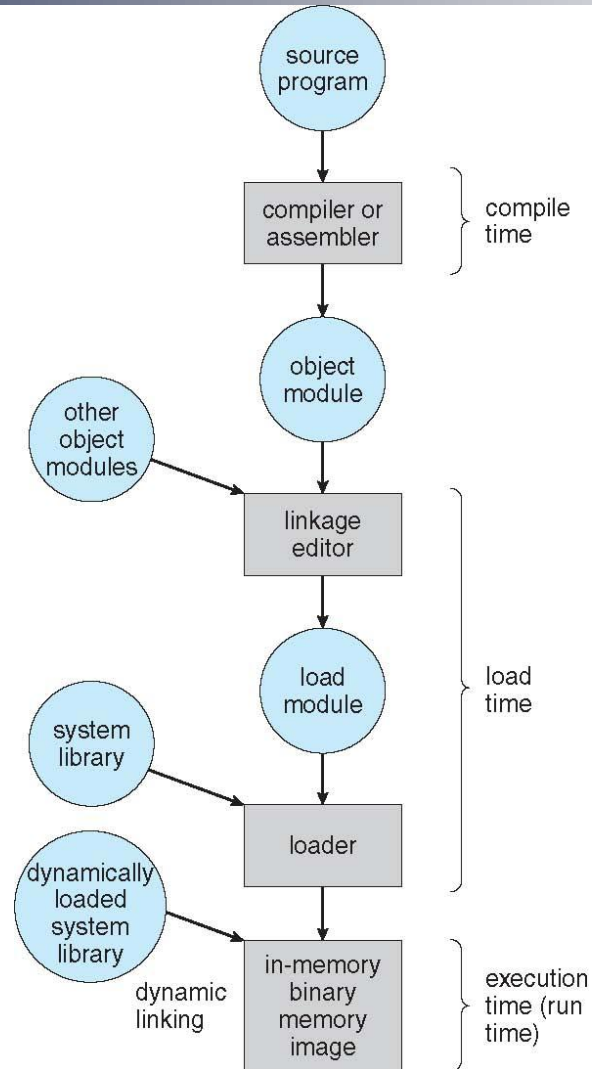


주소 보호

- 기준(**base**) 및 상한(**limit**) 레지스터에 의한 하드웨어적인 메모리 보호



사용자 프로그램의 단계별 처리 과정



주소 바인딩(Address Binding)

- ❑ 대부분의 시스템은 사용자 프로세스가 메모리 내 어느 부분에 적재되더라도 실행 가능하도록 지원함
- ❑ 앞의 과정에서 주소들은 여러가지 다른 방식으로 표현됨
 - ➡ 소스 코드에서의 주소는 심볼 (변수 **count**)
 - ➡ 컴파일 된 코드에서의 주소는 재배치 가능한 주소
 - ❑ 이 모듈의 시작부터 **14** 바이트 뒤
 - ➡ 링커 또는 로더는 재배치 가능 주소를 절대 주소에 바인딩 (예를 들어 **74,014** 번지)
 - ➡ 즉, 각 바인딩은 하나의 주소를 다른 주소로 매핑한다.

주소 바인딩 시점

- 명령과 데이터를 메모리 주소로 바인딩 하는 작업은 3개의 시점에서 가능하다
 - ➡ 컴파일 시간(Compile time): 메모리 위치를 미리 알고 있다면 절대주소(**absolute code**)가 생성될 수 있다. 시작 주소가 변경되면 다시 컴파일 되어야 한다
 - ➡ 적재 시간(Load time): 컴파일 할 때 주소를 알 수 없다면, 재배치 가능(**relocatable code**)가 생성되어야 한다
 - ➡ 실행 시간(Execution time): 프로그램이 실행 중에 메모리 위치를 변경할 수 있다면 바인딩은 실행 시간까지 연기되어야 한다
 - 주소 변환을 위한 하드웨어 지원이 필요

논리 대 물리 주소 공간

- 논리 주소(Logical address)

- ➔ CPU에 의해 생성, 가상 주소(virtual address)로도 불림.

- 물리 주소(Physical address)

- ➔ 메모리 장치에서 인식되는 주소

- 논리 주소와 물리 주소는 컴파일 시간과 적재 시간 바인딩 기법에서는 동일

- 실행 시간 바인딩 기법에서 논리(가상) 주소와 물리 주소는 다름

- 논리 주소 공간(Logical address space)

- ➔ 프로그램에 의해 생성되는 모든 논리 주소의 집합을 의미

- 물리 주소 공간(Physical address space)

- ➔ 프로그램에 의해 생성되는 모든 물리 주소의 집합을 의미

동적 재배치

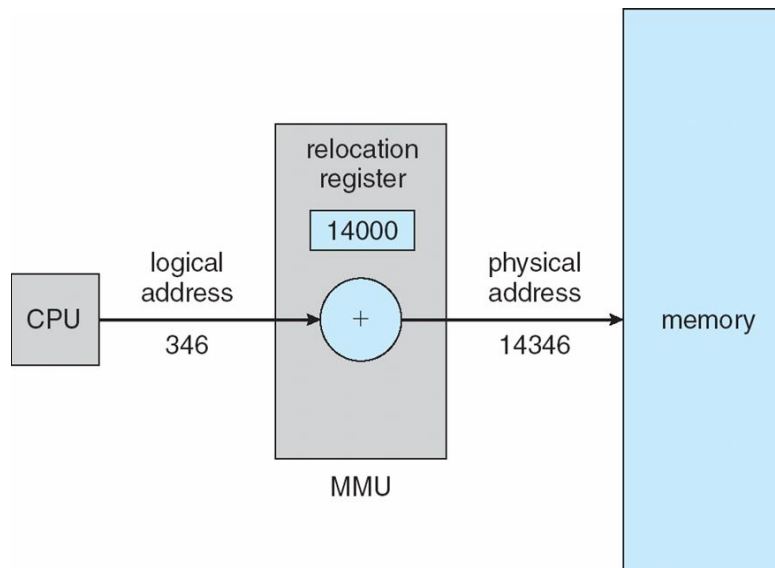
□ MMU(Memory-Management Unit)

➡ 실행 중 가상 주소를 물리 주소로 매핑하는 하드웨어 장치

□ 사용자 프로그램은 논리 주소만을 사용, 실제 물리 주소를 볼 수 없다

➡ 실행 시간 바인딩은 메모리 참조가 일어날 때 바인딩 한다

➡ 논리 주소는 물리 주소로 바인딩된다



동적 링킹 (Dynamic Linking)

❑ 정적 링킹(Static linking)

- ➔ 로더가 시스템 라이브러리와 프로그램 코드를 결합하여 이진 프로그램 이미지 생성

❑ 동적 링킹(Dynamic linking)

- ➔ 실행 시간까지 링킹이 연기됨, 시스템 라이브러리에 주로 사용

❑ 스텝(stub)

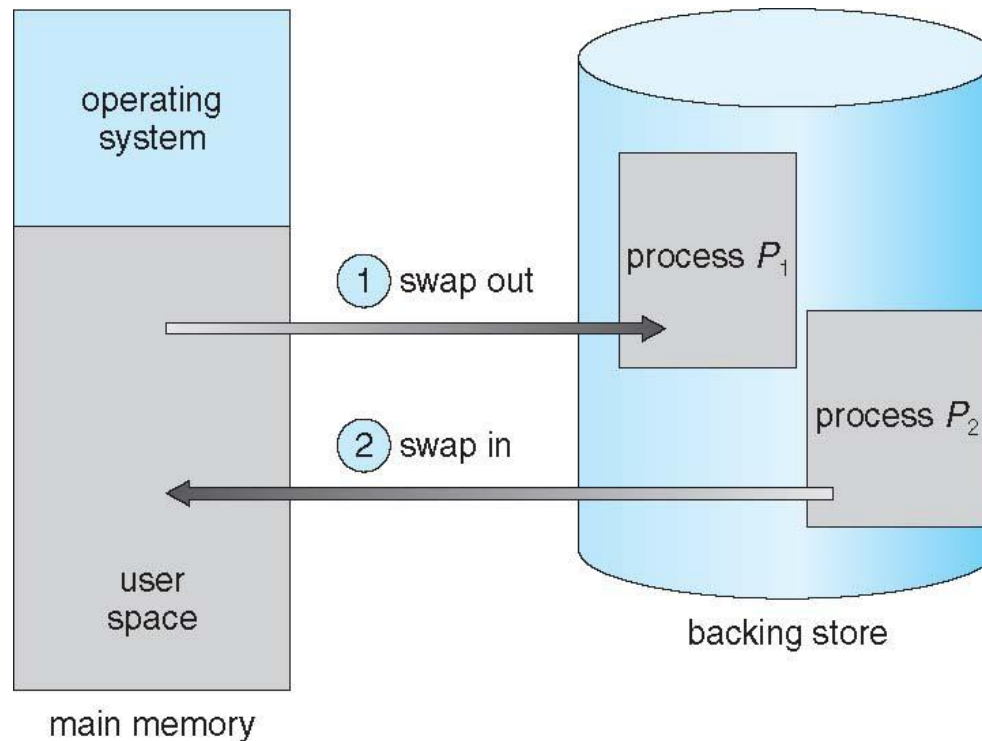
- ➔ 동적 링킹에서 라이브러리 호출 위치에 생성
- ➔ 라이브러리 호출을 대체하는 작은 코드로 라이브러리 루틴의 위치를 찾기 위해 사용
- ➔ 호출 라이브러리 루틴이 메모리에 이미 존재하는지 검사, 존재하지 않는다면 디스크에서 가져옴
- ➔ 자기자신을 루틴의 주소로 대체하고 루틴을 실행
- ➔ 예) `printf()` 루틴은 10개의 프로세스가 사용하더라도, 메모리 상에는 하나만 존재 가능

공유 라이브러리(shared libraries)

- ❑ 동적 링크를 사용하는 라이브러리를 공유 라이브러리(shared libraries)라고도 함
- ❑ 라이브러리 업그레이드에도 유용
 - ➡ 공유 라이브러리 사용하는 프로그램은 라이브러리가 자동 업그레이드 됨

7.2 스와핑(Swapping)

- 프로세스는 임시적으로 메모리에서 보조 저장장치(backing store)로 스왑될 수 있고, 실행을 재개하기 위해 메모리로 다시 반입될 수 있음



스와핑을 포함한 문맥 교환 시간

- ❑ 다음에 실행시킬 프로세스가 메모리에 없다면 프로세스 하나를 스왑-아웃 시키고 목표 프로세스를 스왑-인 해야 한다
- ❑ 이 경우, 문맥 교환 시간이 매우 길어질 수 있음
- ❑ 100MB 크기의 프로세스를 전송률 50MB/sec로 디스크에 스왑 아웃 하는 경우
 - ➡ 스왑-아웃 시간: 2000 ms
 - ➡ (같은 크기의) 프로세스를 스왑-인 하는 시간
 - ➡ 문맥 교환 시간 중 스와핑 시간은 4000ms (4 seconds)

스와핑에 대한 다른 제약

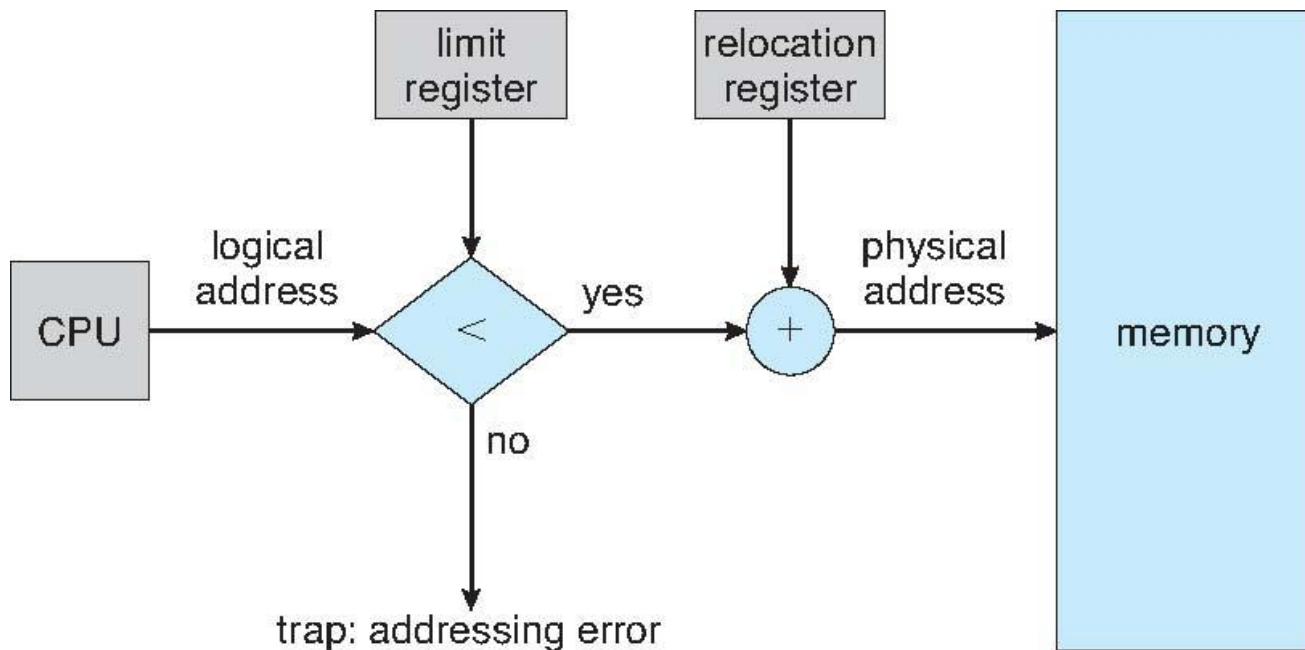
- ❑ 스와핑 대상 프로세스는 **IDLE** 상태인 프로세스
- ❑ **I/O 대기 중인 프로세스**
 - ➡ 입출력이 다른 프로세스의 주소 공간에서 일어나기 때문에 스왑-아웃 할 수 없음
- ❑ 해결책
 - ➡ I/O 동작은 프로세스와 직접 수행하지 않고, 운영체제 버퍼 공간으로 전달, 이 후 운영체제가 해당 프로세스에게 전달
 - ➡ 이러한 이중 버퍼링(double buffering) 자체가 오버헤드
- ❑ 이러한 표준 스와핑은 현대 운영체제에서는 사용되지 않음
 - ➡ 변형 스와핑
 - ❑ 가용 메모리가 특정 임계치보다 적을 경우만 스와핑 발생
 - ❑ 변형 스와핑은 일부 운영체제에서 사용함

7.3 연속 메모리 할당

- ❑ 운영체제의 초기 메모리 할당 방식
- ❑ 프로세스 전체를 메모리의 연속적인 공간에 배치하는 방식
- ❑ 메인 메모리는 2개로 분할
 - ➡ 상주하는 운영체제 프로세스들, 보통 인터럽트 벡터와 함께 메모리 앞쪽에 위치한다
 - ➡ 사용자 프로세스는 그 이후 영역에 배치한다
 - ➡ 각 프로세스는 하나의 연속된 메모리 영역에 저장

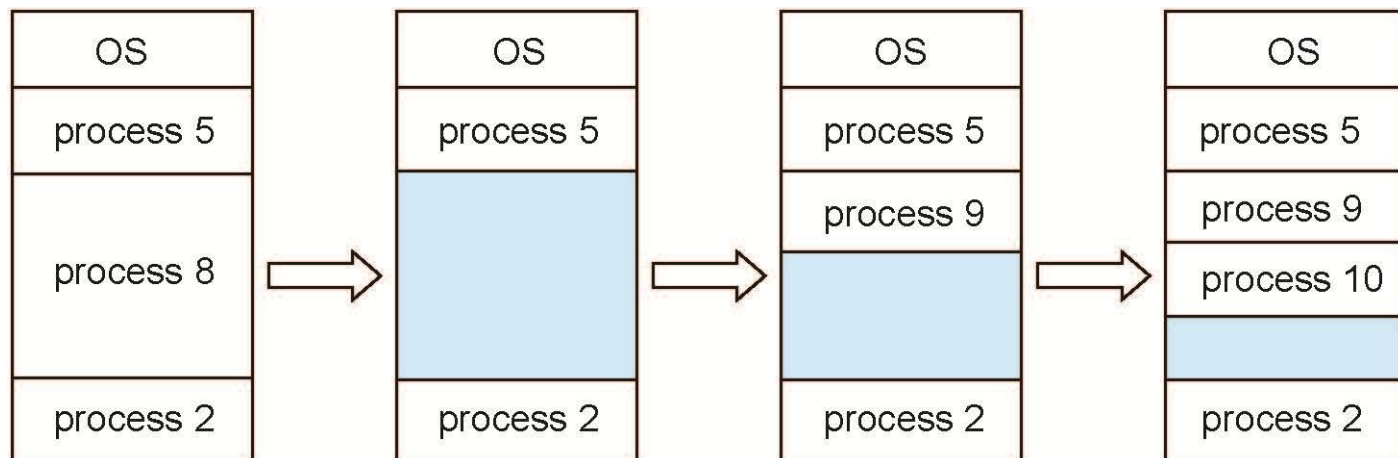
연속 할당에서 메모리 매핑

- 운영체제 프로세스와 각 사용자 프로세스의 보호를 위해 재배치 레지스터 사용



다중 분할 및 가변 분할 할당

- ❑ 분할의 개수가 다중 프로그래밍의 정도를 제한
- ❑ 가변 분할(Variable-partition)
 - ➡ 프로세스의 요청에 맞추어 분할의 크기를 조절
- ❑ 가용 공간(Hole)
 - ➡ 가용 메모리 블록, 다양한 크기의 hole이 메모리 전체에 흩어진다
- ❑ 프로세스가 도착할 때 프로세스를 모두 포함할 만큼의 큰 hole에서 메모리를 할당 받는다
- ❑ 종료되는 프로세스는 자신의 분할을 반환하고 인접한 가용 분할과 합쳐진다



동적 메모리 할당 문제

- 크기 n 인 요청을 만족시키는 hole을 어떻게 찾을 것인가?
 - ➡ 최초 적합(First-fit): 충분히 큰 최초의 hole을 할당
 - ➡ 최적 적합(Best-fit): 충분히 큰 가장 작은 hole을 할당, 크기 순으로 정렬되어 있지 않다면 리스트 전체를 탐색해야 함
 - 할당 결과, 가장 작은 hole을 생성 확률이 높음
 - ➡ 최악 적합(Worst-fit): 가장 큰 hole을 할당, 역시 리스트 전체를 탐색해야 함
 - 할당 결과, 가장 큰 hole 생성 확률이 높음
- 시뮬레이션 등의 연구 결과, 최초 적합과 최적 적합이 최악 적합 보다 속도와 공간 이용률 측면에서 더 좋다는 것이 입증됨

단편화 (Fragmentation)

□ 외부 단편화(External Fragmentation)

- ➡ 가용 공간의 크기를 모두 합하면 요청을 만족시킬 수 있지만 연속적이지 않은 상황

□ 내부 단편화(Internal Fragmentation)

- ➡ 할당된 메모리가 요청된 크기 보다 살짝 큰 경우로 내부적으로 공간이 있지만 사용되지 않는 상황

□ 최초 적합 방식에 대해 통계적 분석해 결과, N blocks이 할당되면, $0.5N$ blocks은 단편화로 사용할 수 없게 된다

- ➡ $1/3$ 이 사용할 수 없을 수 있다 -> 50-percent rule

단편화 해결책

- ❑ 외부 단편화는 밀집(**compaction**) 작업으로 줄일 수 있다
 - ➡ 메모리 내용을 이동시켜 모든 가용 메모리가 하나의 커다란 블록이 되도록 만든다
 - ➡ 밀집은 재배치가 동적이고 실행 시간에 이루어질 때만 가능하다
 - ➡ I/O 문제
 - ❑ 입출력이 진행 프로세스의 재배치는 불가능

7.4 세그먼테이션(Segmentation)

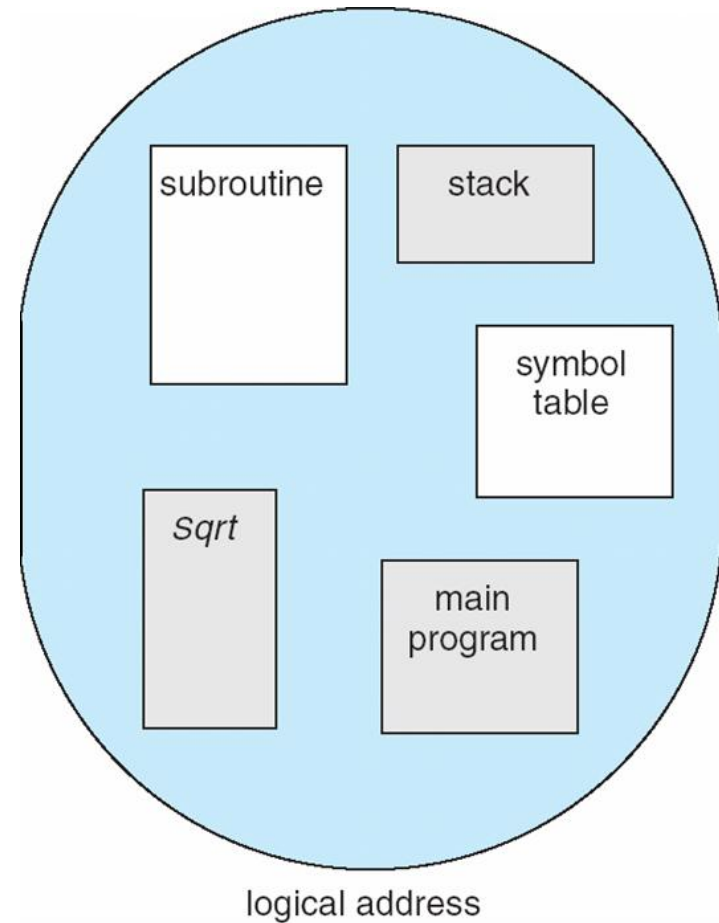
□ 비 연속 메모리 할당 방법

- ➔ 하나의 프로세스 메모리가 연속적인 메모리 공간에 할당되지 않고, 세그먼트라는 프로그래머가 생각하는 단위로 저장

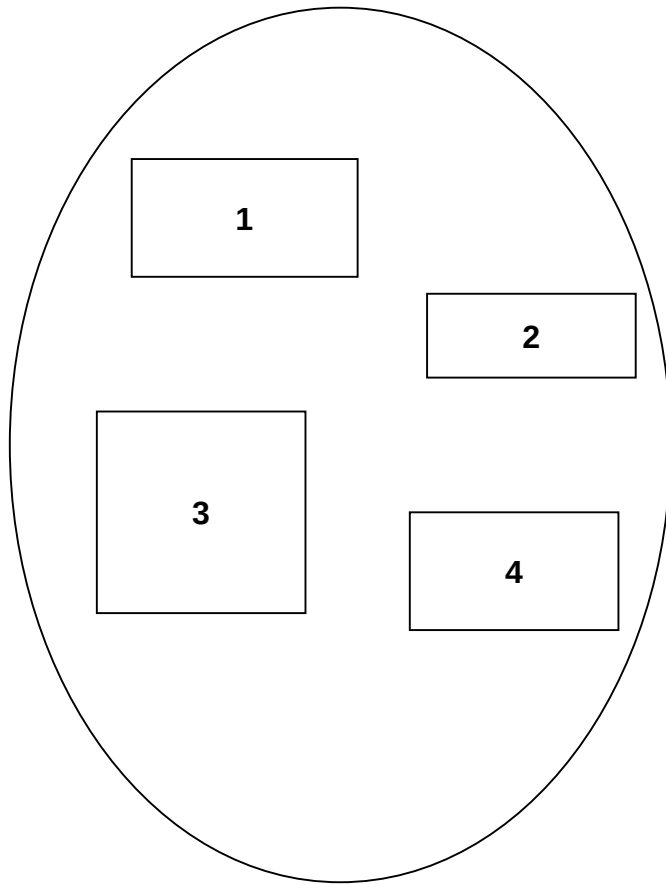
□ 프로그래머가 생각하는 프로그램 모습 (오른쪽 그림)

□ C 프로그램 세그먼테이션 예

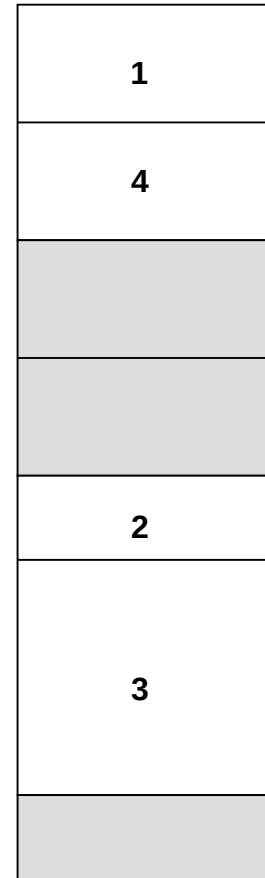
- ➔ 코드
- ➔ 전역 변수
- ➔ 힙 (동적 할당)
- ➔ 스택
- ➔ 표준 C 라이브러리



세그먼테이션의 논리적 관점



user space



physical memory space

세그먼트 구조

- 논리 주소는 2-tuple로 구성

<segment-number, offset>

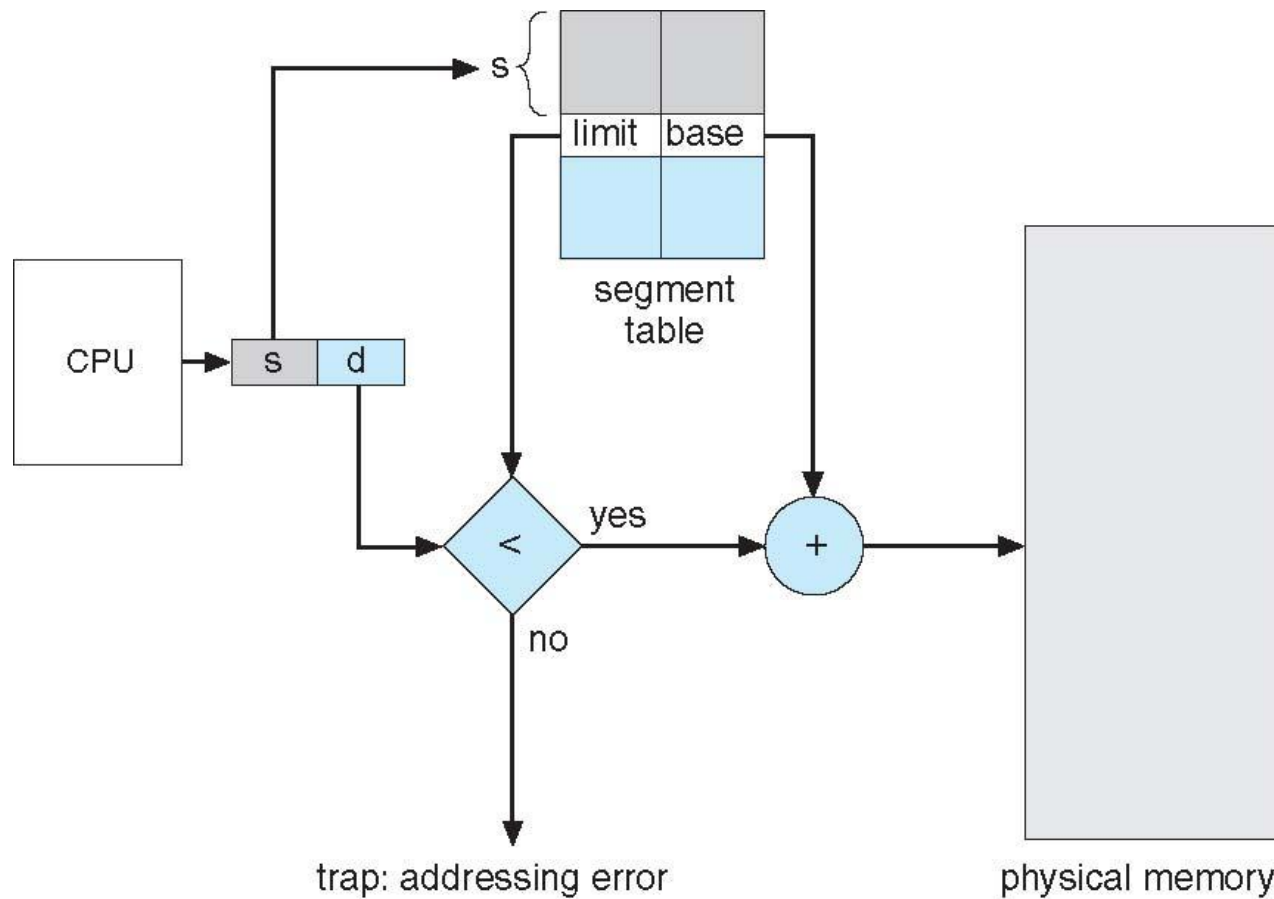
- 세그먼트 테이블(Segment table): 물리 주소를 매핑

- ➔ base: 세그먼트가 위치하는 메모리의 시작 물리 주소
- ➔ limit: 세그먼트의 길이를 지정

- 보호

- ➔ 유효 비트
- ➔ Read/Write/Execute 권한 (공유 지원)

세그먼테이션 하드웨어



세그먼테이션 장/단점

□ 장점

- ➔ 프로그램 구조 반영
- ➔ 프로세스 간의 공유 및 보호가 우수

□ 단점

- ➔ 외부 단편화 발생 위험이 큼
 - 세그먼트 크기가 서로 다르기 때문에 메모리 중간에 **hole** 발생 확률이 높음
- ➔ 메모리 밀집(**compaction**) 필요
- ➔ 관리 복잡성이 커짐

□ 현재 운영체제에서 세그먼테이션만 단독으로 사용하는 경우는 없음

- ➔ 가상 메모리 구현에 페이징(7.5절)과 같이 활용

7.5 페이징 (Paging)

동일 크기

□ 프로세스 메모리 공간을 일정 크기 단위로 나누어 관리하는 형태의 불연속 메모리 할당 방법

➔ 외부 단편화 현상을 감소

➔ 가변 길이 메모리 단위(세그멘테이션)의 관리 복잡성 감소

□ 메모리 분할 단위

➔ 논리 메모리: 페이지로 구성

➔ 물리 메모리: 프레임으로 구성

➔ 분할 크기: 512 bytes ~ 16 Mbytes (2ⁿ 단위 크기)

□ 예: $2^{12} = 4\text{KB}$

➔ 페이지와 프레임 크기는 동일

□ N 개의 페이지로 구성된 프로그램을 실행시키기 위해서 N 개의 가용 프레임을 찾아 적재해야 한다

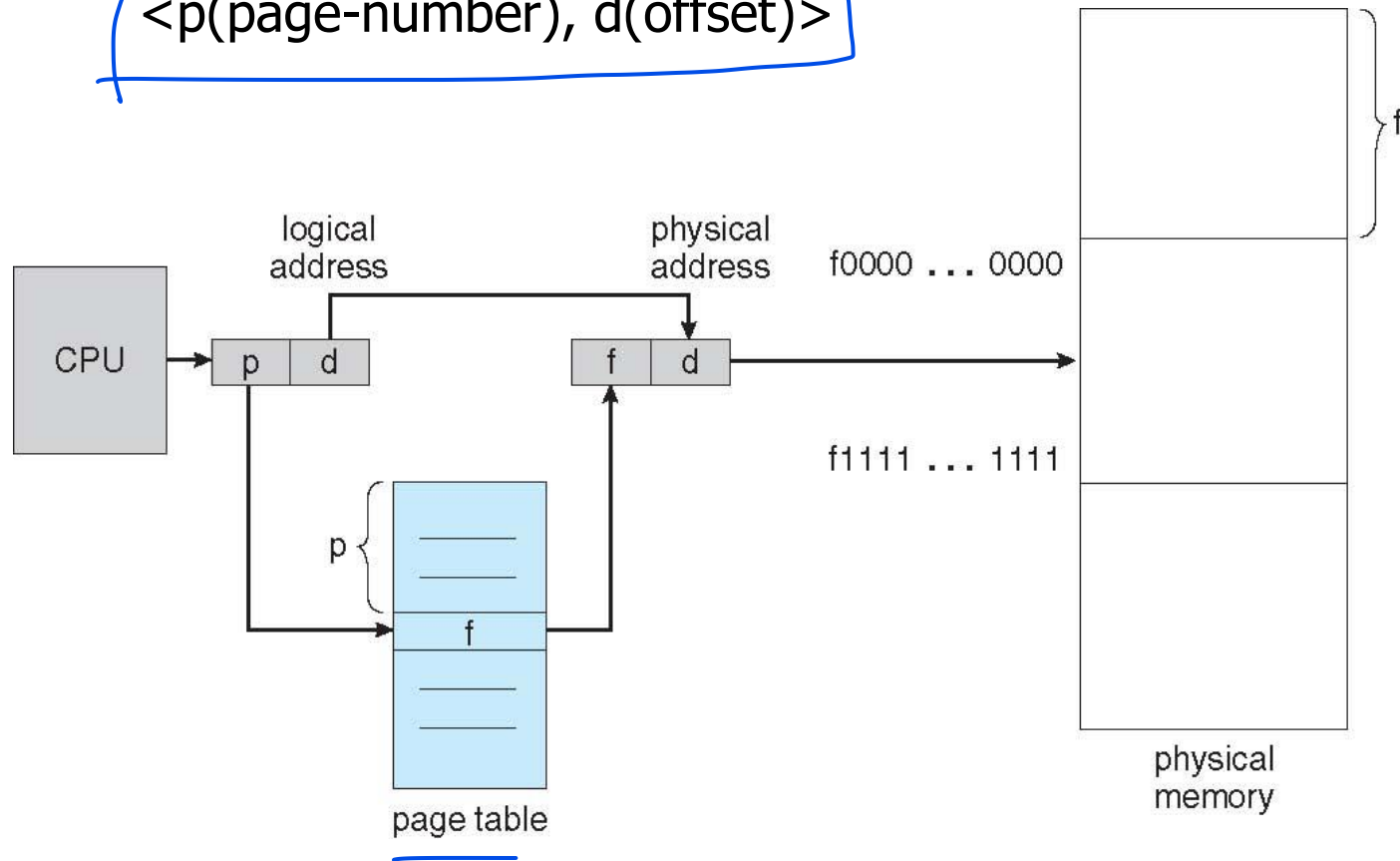
□ 내부 단편화 가능성은 여전히 존재

외부 단편화를 완화하기 위해 분할을 줄이는건데 너무 줄이면 페이징의 장점을 못챌 수 있음.

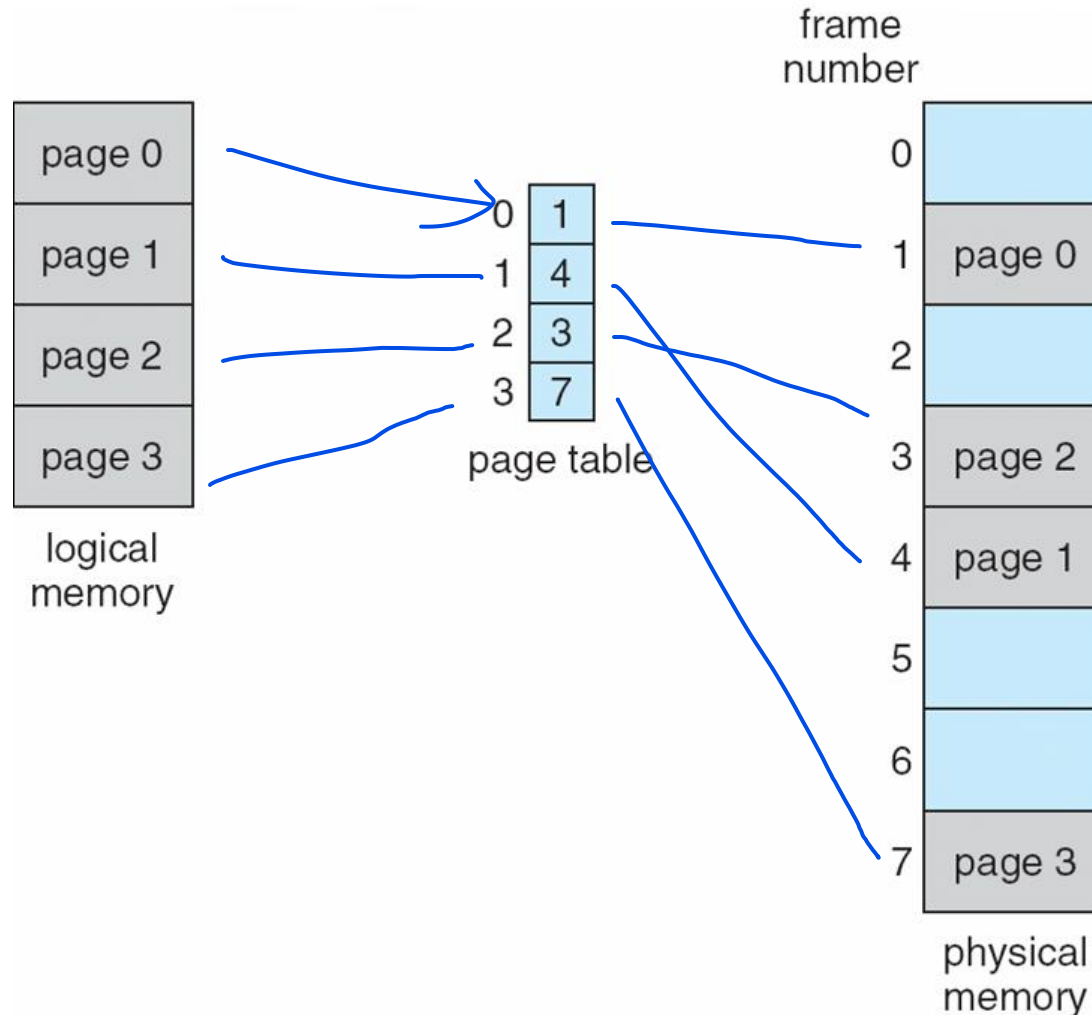
페이징 하드웨어 그림 이해하기

- 논리 주소는 **2-tuple**로 구성

$\langle p(\text{page-number}), d(\text{offset}) \rangle$



논리 메모리와 물리 메모리의 페이징 모델



주소 변환 기법

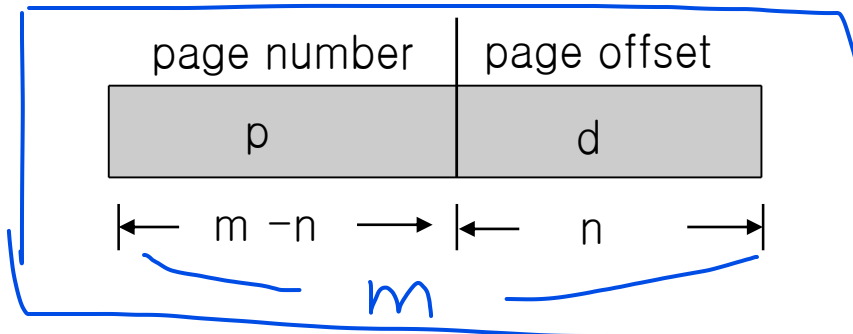
논리 주소

□ CPU에 의해 생성된 주소는 다음과 같은 부분으로 나뉘어진

다: $\langle p, d \rangle$

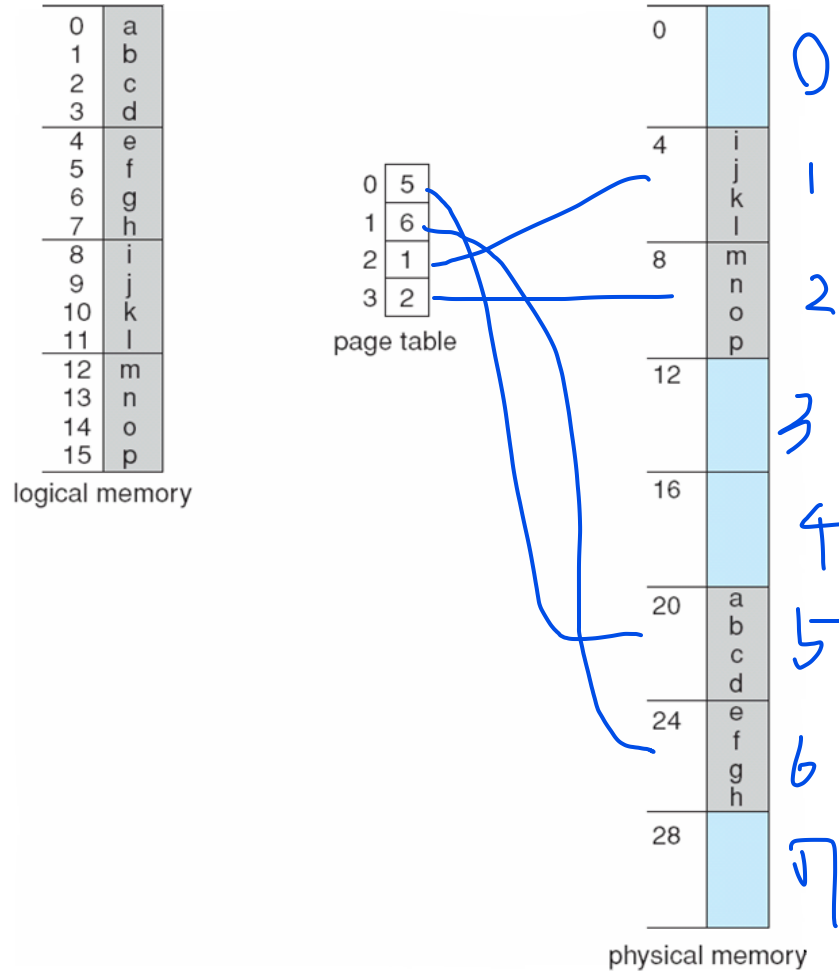
- ➔ **Page number (p)** – 물리 메모리 페이지의 시작 주소를 저장하고 있는 페이지 테이블의 인덱스로 사용됨
- ➔ **Page offset (d)** – 기준 주소와 결합되어 메모리 장치로 보내지는 물리 주소를 정의함

➔ 논리 주소 공간의 크기가 2^m 이고 페이지 크기가 2^n



페이징 예제

□ $n=2$ 와 $m=4$, 32-byte 메모리와 4-byte 페이지



내부 단편화

□ 내부 단편화 예

➔ 페이지 크기 = 2,048 bytes *2K byte*

➔ 프로세스 크기 = 72,766 bytes

➔ 35 pages + 1,086 bytes $\Rightarrow$ 36페이지를 할당해야함

➔ 내부 단편화 $2,048 - 1,086 = 962$ bytes $\rightarrow$ 962 bytes가 남고, 이 부분을 사용할 수 없음

□ 최악의 경우 단편화 = 1 frame - 1 byte

□ 평균적으로 1 / 2 frame size

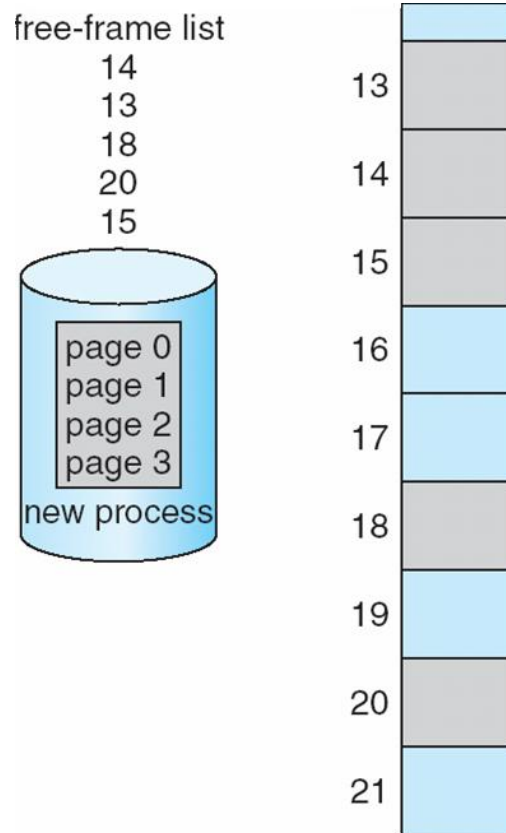
□ 그렇다면 작은 크기의 프레임이 바람직한가?

➔ 그러나 각 페이지 테이블 항목의 개수가 많아지면 메모리를 많이 차지하게 된다 $\Rightarrow$ 즉, 페이지의 장점이 줄어든다.

□ 페이지 크기는 점점 커지고 있다

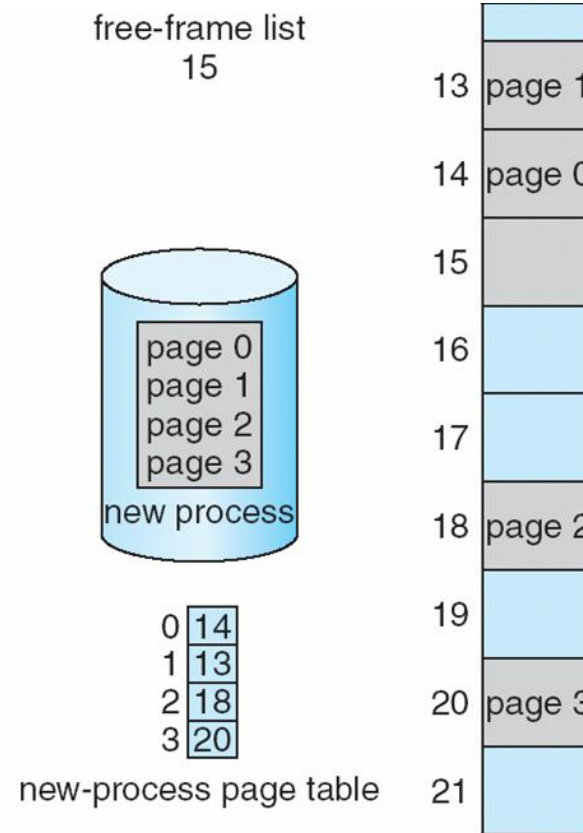
➔ Solaris는 2개 크기의 페이지 지원 - 8 KB and 4 MB

가용 프레임



(a)

Before allocation



(b)

After allocation

페이지 테이블

□ 시스템에서 페이지 테이블 접근은 빠른 속도를 지원해야 함

➔ 일부 시스템은 페이지 테이블을 전용 레지스터로 구현

□ 매우 작은 크기의 페이지 테이블만 지원 가능

특수한 경우

□ 대부분의 시스템에서 페이지 테이블 구현 방법

➔ PTBR (Page-Table Base Register) 도입: 페이지 테이블의 위치를 빠르게 참조 가능하다.

➔ TLB (Translation Look-aside Buffers) 캐시 도입

□ TLB

➔ 캐시 메모리로 구현

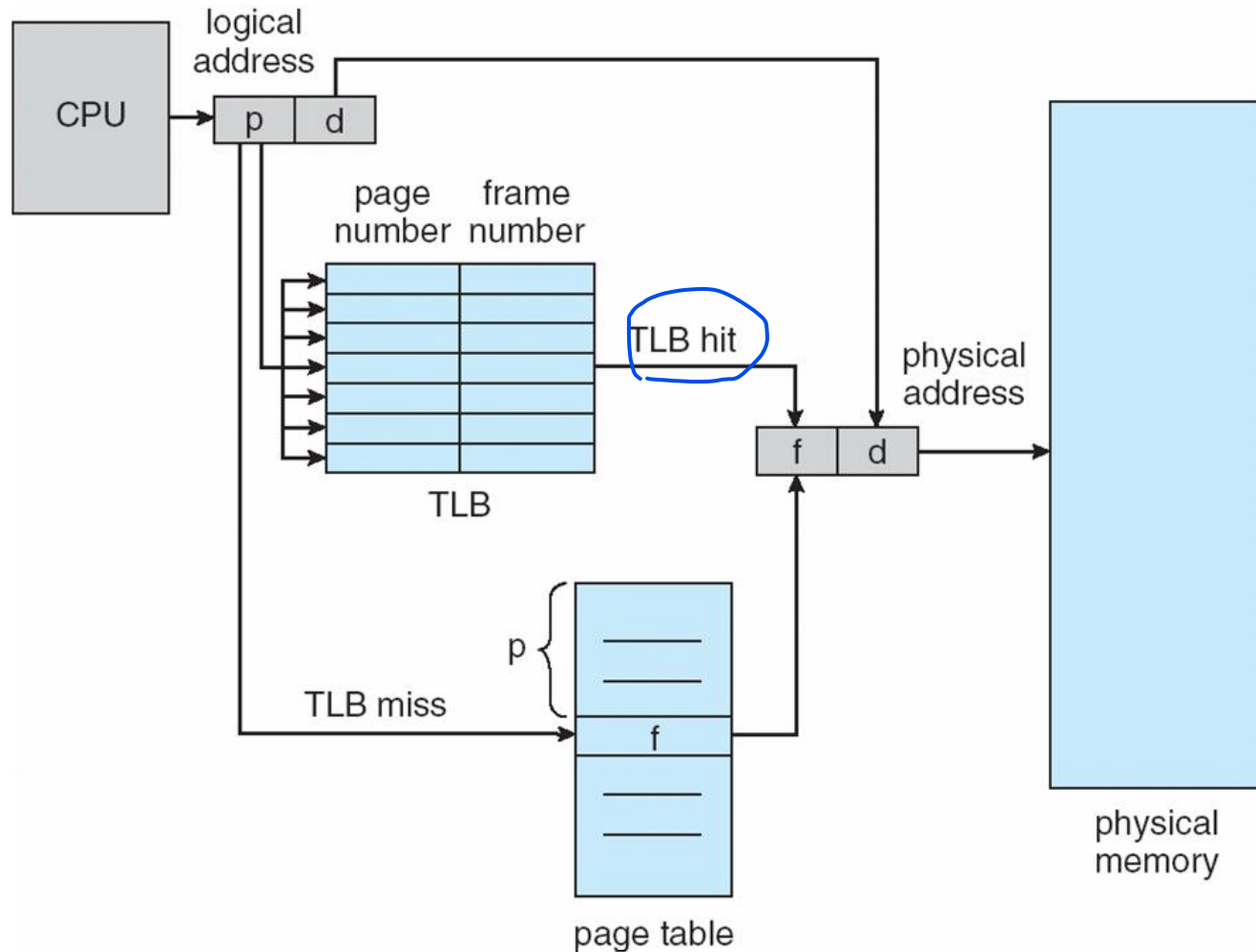
➔ 페이지 테이블의 일부를 저장

□ 보통 32 ~ 1024 개 항목 저장

➔ TLB 참조는 일반적으로 CPU 내 파이프라인 명령어로 처리

□ 거의 성능 저하가 없음

TLB를 활용한 페이징 하드웨어



TLB 구성 및 ASID

□ TLB 구성 항목

- ➔ Virtual Page Number: 가상 페이지 번호
 - ➔ Physical Frame Number: 물리 프레임 번호
 - ➔ Valid bit: 유효 여부
 - ➔ Protection bit: 접근 권한
 - ➔ Reference bit: 참조 여부
 - ➔ ASID: 주소 공간 식별자
- 주
부수
공유나 보호

□ ASID (Address Space ID)(PID)

- ➔ TLB 엔트리가 어떤 프로세스 소속인지를 나타내는 식별자
- ➔ 일부 시스템에서는 서로 다른 프로세스가 동일한 가상 페이지 번호를 사용할 수 있음
 - TLB 검색은 <VPN, ASID>로 수행
- ➔ Context Switch 시 TLB flush 감소 (효율 증가)

TLB 관리

- TLB는 일종의 캐시이므로 교체 정책 등의 관리가 필요
 - ➡ TLB 미스가 발생하면, 그 항목은 TLB 항목에 추가
 - ➡ 교체 정책: LRU(Least Recently Used), 라운드 로빈, 무작위 등
 - ➡ 일부 TLB 항목은 고정
 - 중요 커널 코드 등

TLB 적중률(hit ratio)와 유효 접근 시간

- 메모리 접근 시간: 100 ns, TLB 적중률: 80%
 - ➔ 유효 접근 시간 = $0.8 * 100 + 0.2 * 200 = 120$ ns
- 메모리 접근 시간: 100 ns, TLB 적중률: 99%
 - ➔ 유효 접근 시간 = $0.99 * 100 + 0.01 * 200 = 101$ ns
- 일부 CPU는 여러 단계의 TLB를 사용
 - ➔ L1 TLB, L2 TLB

메모리 보호

- ❑ 메모리 보는 각 프레임과 연관된 보호 비트를 통하여 구현된다. 보호 비트는 읽기 전용, 읽고 쓰기가 허용되는지 표시한다
 - ➡ 실행만 가능 등을 표시하는 추가적인 비트를 더할 수도 있다
- ❑ 유효-무효(Valid-invalid) 비트는 페이지 테이블의 각 항목에 포함된다
 - ➡ “valid”는 연관된 페이지가 프로세스의 논리 주소 공간에 속하며 따라서 합법적인 페이지임을 나타낸다
 - ➡ “invalid”는 페이지가 논리 주소 공간에 존재하지 않는다는 것을 나타낸다
 - ➡ 또는 page-table length register (PTLR)을 사용한다
- ❑ 보호에 대한 위반이 발생하면 커널로 트랩된다

Valid (v) or Invalid (i) Bit In A Page Table

00000	page 0
	page 1
	page 2
	page 3
	page 4
10,468	page 5
12,287	

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page <i>n</i>

공유 페이지

□ 공유 코드(Shared code)

- ➡ 하나의 읽기 전용 복사본(reentrant) code를 프로세스들이 공유 (i.e., text editors, compilers, window systems)
- ➡ 같은 프로세스 공간을 공유하는 다수의 스레드와 유사
- ➡ 읽기-쓰기 페이지를 공유한다면 프로세스간 통신에 유용

□ Private code and data

- ➡ 각 프로세스는 코드와 데이터의 별도의 사본을 유지한다
- ➡ 이 페이지들은 논리 주소 공간의 아무데나 존재할 수 있다

공유 페이지 예제

